

PART XIII

GRAPHS

Representations, traversal, shortest paths, spanning trees, and DSU

Graphs are the most general data structure in this book — an array is a graph where every node points only to the next one, and a tree is a graph with no cycles. This Part is intentionally one of the largest: it starts from raw terminology and builds all the way to shortest paths, spanning trees, and the connectivity toolkit (DSU, SCCs, bridges) that Codeforces and LeetCode both lean on constantly. Take it in stages — traversal first, then shortest paths, then MST/DSU, then the advanced section — rather than end to end in one sitting.

Contents of Part XIII

- 1. Graph Terminology and Representations
- 2. Breadth-First Search (BFS)
- 3. Depth-First Search (DFS)
- 4. Connected Components and Cycle Detection
- 5. Bipartite Graphs
- 6. Topological Sort
- 7. Shortest Paths I — BFS Shortest Path and 0-1 BFS
- 8. Shortest Paths II — Dijkstra's Algorithm
- 9. Shortest Paths III — Bellman-Ford and Floyd-Warshall
- 10. Minimum Spanning Tree — Kruskal and Prim
- 11. Disjoint Set Union (DSU)
- 12. Advanced Graph Topics — SCC, Bridges, Articulation Points
- Part XIII Revision, Cheat Sheet, and Interview Questions

Chapter 1 — Graph Terminology and Representations

1.1 What is a graph?

In one sentence: a graph is a set of nodes (vertices) connected by edges, representing pairwise relationships — with no restriction on shape, unlike a tree.

A tree is just a special graph: connected, with no cycles, and exactly $n-1$ edges for n nodes. A graph drops all three restrictions — it can be disconnected, it can have cycles, and it can have any number of edges.

1.2 Core vocabulary

Term	Meaning
Vertex / node	A single point in the graph
Edge	A connection between two vertices
Directed edge	$u \rightarrow v$ can be traversed only from u to v
Undirected edge	$u - v$ can be traversed both ways
Weighted graph	Each edge carries a cost/length
Degree (undirected)	Number of edges touching a vertex
In-degree / out-degree (directed)	Number of incoming / outgoing edges
Path	A sequence of vertices connected by edges, no repeats
Cycle	A path that returns to its starting vertex
Connected component	A maximal set of vertices reachable from each other (undirected)

1.3 Adjacency list

The default representation for almost all competitive programming: for each vertex, store a list of its neighbors.

```

int n, m; // n vertices, m edges
cin >> n >> m;
vector<vector<int>> adj(n + 1); // 1-indexed; adj[u] = list of
neighbors of u

for (int i = 0; i < m; i++) {
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u); // omit this line for a DIRECTED graph
}

// weighted graph: store (neighbor, weight) pairs
vector<vector<pair<int,int>>> wadj(n + 1);
wadj[u].push_back({v, weight});

```

Representation	Space	Check edge (u,v)	Iterate neighbors of u	Best for
Adjacency matrix	$O(V^2)$	$O(1)$	$O(V)$	Dense graphs, small V, frequent edge lookups
Adjacency list	$O(V + E)$	$O(\text{degree}(u))$	$O(\text{degree}(u))$	Sparse graphs — the CP default
Edge list	$O(E)$	$O(E)$	$O(E)$	Algorithms that process all edges at once (Kruskal, Bellman-Ford)

⚠ Common mistake

Using an adjacency matrix by habit for a graph with up to 10^5 vertices. $O(V^2)$ memory at $V = 10^5$ is 10^{10} entries — far beyond any memory limit. Default to an adjacency list unless V is small (roughly $V \leq 2000$) or the problem specifically needs $O(1)$ edge lookups.

1.4 Directed vs. undirected, weighted vs. unweighted

These two independent choices — direction and weight — determine which algorithm applies. An unweighted graph is really a weighted graph where every edge has weight 1; BFS (Chapter 2) exploits exactly

that fact to find shortest paths without needing Dijkstra at all.

Chapter 2 — Breadth-First Search (BFS)

In one sentence: BFS explores a graph level by level, visiting all vertices at distance 1 from the start before any vertex at distance 2 — using a queue to enforce that order.

```
vector<int> bfs(int start, vector<vector<int>> &adj, int n) {
    vector<int> dist(n + 1, -1); // -1 = unvisited
    queue<int> q;
    dist[start] = 0;
    q.push(start);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) { // not visited yet
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
    return dist; // dist[v] = shortest #edges from start to v, or -1
                // if unreachable
}
```

Dry run: BFS from vertex 1 on 1-2, 1-3, 2-4, 3-4, 4-5

Step	Queue after step	dist[] so far (1..5)
Start	[1]	[0, -1, -1, -1, -1]
Pop 1, visit 2, 3	[2, 3]	[0, 1, 1, -1, -1]
Pop 2, visit 4	[3, 4]	[0, 1, 1, 2, -1]
Pop 3, 4 already visited	[4]	[0, 1, 1, 2, -1]
Pop 4, visit 5	[5]	[0, 1, 1, 2, 3]

Complexity: $O(V + E)$ — every vertex is enqueued at most once, and every edge is examined at most once (twice for an undirected graph, once from each endpoint).

🔗 Why BFS gives shortest paths in unweighted graphs

BFS only ever moves to a vertex the first time it's reached, and it processes vertices in non-decreasing order of distance from the start — by the time a vertex is dequeued, every shorter path to it has already been considered. This is the proof sketch behind an interview-favorite question; be ready to state it in words, not just cite the result.

Practice — Chapter 2**Problem 13.1 — Shortest Path in an Unweighted Grid [Level 2]**

Hint 1: Treat each grid cell as a node, with edges to its up/down/left/right neighbors.

Approach: Run BFS from the start cell; the distance to the target cell is the answer.

Complexity: $O(\text{rows} \times \text{cols})$.

Problem 13.2 — Rotting Oranges (multi-source BFS) [Level 3]

Hint 1: There can be more than one starting rotten orange.

Key observation: Pushing ALL initially-rotten oranges into the queue before starting BFS handles multiple sources for free — no separate BFS per source needed.

Approach: Multi-source BFS; track the maximum distance reached.

Complexity: $O(\text{rows} \times \text{cols})$.

Chapter 3 — Depth-First Search (DFS)

In one sentence: DFS explores as far as possible down one path before backtracking — implemented either with real recursion (the call stack) or an explicit stack.

```
vector<bool> visited;
vector<vector<int>> adj;

void dfs(int u) {
    visited[u] = true;
    // process u here (pre-order)
    for (int v : adj[u]) {
        if (!visited[v]) dfs(v);
    }
}

// iterative version using an explicit stack (avoids
// recursion-depth limits)
void dfsIterative(int start, int n) {
    vector<bool> vis(n + 1, false);
    stack<int> st;
    st.push(start);
    while (!st.empty()) {
        int u = st.top(); st.pop();
        if (vis[u]) continue;
        vis[u] = true;
        // process u here
        for (int v : adj[u]) if (!vis[v]) st.push(v);
    }
}
```

⚠ Common mistake

Plain recursive DFS on a graph with up to 10^5 – 10^6 vertices can overflow the call stack (a chain-shaped graph gives recursion depth = V). Prefer the iterative version, or raise stack size explicitly, whenever V is large and the graph could be a long chain.

BFS vs. DFS: both are $O(V + E)$ and both visit every reachable vertex, but they explore in a different order. Use BFS whenever “shortest” or “fewest steps” matters (unweighted shortest path, level-by-level structure); use DFS for connectivity questions, cycle detection, topological sort, and anywhere the order of full exploration — not distance — is what matters.

Chapter 4 — Connected Components and Cycle Detection

4.1 Counting connected components

Run DFS (or BFS) from every unvisited vertex; each fresh start covers exactly one connected component.

```
int countComponents(int n, vector<vector<int>> &adj) {
    vector<bool> visited(n + 1, false);
    int components = 0;
    for (int i = 1; i <= n; i++) {
        if (!visited[i]) {
            components++;
            dfsMark(i, adj, visited); // marks the whole component visited
        }
    }
    return components;
}
```

4.2 Cycle detection — undirected graph

During a DFS on an undirected graph, finding an edge to an already-visited vertex that is not the immediate parent means a cycle exists — the parent check matters because every undirected edge is stored twice ($u \rightarrow v$ and $v \rightarrow u$).

```
bool hasCycleUtil(int u, int parent, vector<vector<int>> &adj,
vector<bool> &vis) {
    vis[u] = true;
    for (int v : adj[u]) {
        if (!vis[v]) {
            if (hasCycleUtil(v, u, adj, vis)) return true;
        } else if (v != parent) {
            return true; // visited neighbor that isn't the parent -> cycle
        }
    }
    return false;
}
```

4.3 Cycle detection — directed graph

The parent trick does not work for directed graphs (an edge to a visited vertex can be perfectly legal if that vertex is not an ancestor in

the current DFS path). Instead, track which vertices are on the current recursion stack.

```
bool hasCycleDirected(int u, vector<vector<int>> &adj,
    vector<bool> &vis, vector<bool> &onStack) {
    vis[u] = true;
    onStack[u] = true;
    for (int v : adj[u]) {
        if (!vis[v]) {
            if (hasCycleDirected(v, adj, vis, onStack)) return true;
        } else if (onStack[v]) {
            return true; // back edge to a vertex on the current path ->
cycle
        }
    }
    onStack[u] = false; // done exploring u, remove it from the
current path
    return false;
}
```

⚠ Common mistake

Reusing the undirected “parent” check on a directed graph, or forgetting to clear `onStack[u]` when backtracking. Both produce false positives or false negatives on graphs that mix converging paths (a DAG “diamond” shape) with genuine cycles.

Chapter 5 — Bipartite Graphs

In one sentence: a graph is bipartite if its vertices can be split into two groups such that every edge connects a vertex in one group to a vertex in the other — equivalently, if it can be properly 2-colored.

Check this with BFS or DFS: color the start vertex 0, then color every neighbor the opposite color of its parent. If you ever find an edge between two same-colored vertices, the graph is not bipartite.

```
bool isBipartite(int n, vector<vector<int>> &adj) {
    vector<int> color(n + 1, -1);
    for (int start = 1; start <= n; start++) {
        if (color[start] != -1) continue;
        color[start] = 0;
        queue<int> q; q.push(start);
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (int v : adj[u]) {
                if (color[v] == -1) {
                    color[v] = color[u] ^ 1; // opposite color
                    q.push(v);
                } else if (color[v] == color[u]) {
                    return false; // same color on both ends of an edge
                }
            }
        }
    }
    return true;
}
```

Complexity: $O(V + E)$. Note the outer loop over every vertex — the graph may be disconnected, and each component must be checked and colored independently.

♦ Key fact

A graph is bipartite if and only if it contains no cycle of odd length. You will not usually need to find the cycle explicitly — the 2-coloring BFS/DFS above detects the contradiction directly and is simpler to code.

Chapter 6 — Topological Sort

In one sentence: a topological sort orders the vertices of a Directed Acyclic Graph (DAG) so that every edge $u \rightarrow v$ has u appearing before v — it only exists when the graph has no cycle.

6.1 Kahn's algorithm (BFS-based)

Repeatedly remove a vertex with in-degree 0 (no remaining prerequisites), and decrease the in-degree of its neighbors as it's

removed.

```
vector<int> topoSortKahn(int n, vector<vector<int>> &adj) {
    vector<int> indeg(n + 1, 0);
    for (int u = 1; u <= n; u++)
        for (int v : adj[u]) indeg[v]++;

    queue<int> q;
    for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.push(i);

    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) {
            if (--indeg[v] == 0) q.push(v);
        }
    }
    // if order.size() < n, the graph has a cycle -> no valid
    topological order
    return order;
}
```

✦ Free cycle detection

Kahn's algorithm doubles as a directed-cycle detector: if the resulting order has fewer than n vertices, some vertices never reached in-degree 0, which only happens when they sit inside a cycle. This is a commonly preferred alternative to the onStack DFS method from Chapter 4.

6.2 DFS-based topological sort

Run DFS; when a vertex finishes exploring all its neighbors, push it onto a stack. The final stack, popped top to bottom, is a valid topological order.

```
void dfsTopo(int u, vector<vector<int>> &adj, vector<bool> &vis,
stack<int> &st) {
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) dfsTopo(v, adj, vis, st);
    st.push(u); // push AFTER exploring all descendants
}

vector<int> topoSortDFS(int n, vector<vector<int>> &adj) {
    vector<bool> vis(n + 1, false);
    stack<int> st;
    for (int i = 1; i <= n; i++) if (!vis[i]) dfsTopo(i, adj, vis,
st);
    vector<int> order;
    while (!st.empty()) { order.push_back(st.top()); st.pop(); }
    return order;
}
```

Why does pushing after full exploration work? By the time u is pushed, everything reachable from u is already on the stack below it (pushed earlier) — so reversing the finish order guarantees every vertex appears before everything it points to. Both algorithms are $O(V + E)$; Kahn's is generally preferred in competitive programming because it avoids recursion and naturally detects cycles.

Practice — Chapter 6

Problem 13.3 — Course Schedule [Level 3]

Hint 1: Model each course as a node and each prerequisite as a directed edge.

Key observation: “Can all courses be finished?” is exactly “does this graph have a valid topological order?”

Approach: Kahn's algorithm; if the output order has fewer than n courses, a cycle exists and it's impossible.

Complexity: $O(V + E)$.

Chapter 7 — Shortest Paths I: BFS

Shortest Path and 0-1 BFS

Plain BFS (Chapter 2) already solves shortest paths when every edge has the same weight. This chapter extends that idea one step: what if edges have weight 0 or weight 1 only?

7.1 0-1 BFS

In one sentence: when every edge weighs either 0 or 1, a deque replaces the priority queue Dijkstra would otherwise need — push 0-weight edges to the front and 1-weight edges to the back, keeping the deque sorted by distance without ever calling a comparison-based structure.

```
vector<int> zeroOneBFS(int start, int n,
vector<vector<pair<int,int>>> &adj) {
    // adj[u] = list of (neighbor, weight), weight is 0 or 1
    vector<int> dist(n + 1, INT_MAX);
    deque<int> dq;
    dist[start] = 0;
    dq.push_back(start);
    while (!dq.empty()) {
        int u = dq.front(); dq.pop_front();
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                if (w == 0) dq.push_front(v); // free edge -> explore first
                else dq.push_back(v);
            }
        }
    }
    return dist;
}
```

Complexity: $O(V + E)$ — asymptotically the same as plain BFS, and strictly cheaper than Dijkstra's $O(E \log V)$, because no comparison-based structure is needed.

♦ How to recognize it

Clue words: edge weights restricted to exactly two values (very often 0 and 1) — e.g. “free to switch lanes, costs 1 to change direction.” If you see this pattern, 0-1 BFS is almost always intended over full Dijkstra.

Chapter 8 — Shortest Paths II: Dijkstra's Algorithm

In one sentence: Dijkstra repeatedly picks the unvisited vertex with the smallest known distance, locks it in as final, and relaxes (tries to improve) the distances of its neighbors — using a min-heap to always find that smallest-distance vertex quickly.

This is exactly the min-heap “always expand the current best candidate” pattern from Part XII, Chapter 9 — applied to graph distances instead of a plain array.

```
vector<long long> dijkstra(int start, int n,
vector<vector<pair<int,int>>> &adj) {
    vector<long long> dist(n + 1, LLONG_MAX);
    priority_queue<pair<long long,int>, vector<pair<long long,int>>,
greater<pair<long long,int>>> pq; // min-heap of (distance, node)
    dist[start] = 0;
    pq.push({0, start});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue; // stale entry, a better one was
already processed
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
```

Why the `if (d > dist[u]) continue;` line matters

Because `priority_queue` offers no way to decrease a key in place (Part XII, Chapter 9), a vertex may be pushed more than once with different distances. This line is “lazy deletion”: it skips a popped entry that is now known to be outdated, rather than trying to remove it from the heap directly. Omitting it does not break correctness, but it can process the same vertex many times and hurt performance.

Complexity: $O((V + E) \log V)$ with a binary heap and an adjacency list — each edge can cause at most one push, and each push/pop is $O(\log V)$.

⚠ **Why Dijkstra fails with negative edges**

Dijkstra finalizes a vertex's distance the moment it's popped, assuming no future relaxation could ever improve it — true only when all remaining edge weights are non-negative. A negative edge discovered later could still improve an already-finalized distance, and Dijkstra has no mechanism to revisit it. Use Bellman-Ford (Chapter 9) instead.

Practice — Chapter 8

Problem 13.4 — Network Delay Time [Level 3]

Hint 1: This is single-source shortest path on a weighted directed graph.

Approach: Run Dijkstra from the source; the answer is the maximum finite distance across all nodes (or -1 if any node is unreachable).

Complexity: $O((V + E) \log V)$.

Chapter 9 — Shortest Paths III: Bellman-Ford and Floyd-Warshall

9.1 Bellman-Ford

In one sentence: relax every edge in the graph, $V-1$ times — this correctly handles negative edge weights (unlike Dijkstra) and can detect negative cycles.

```
bool bellmanFord(int start, int n, vector<array<int,3>> &edges,
// {u, v, weight}
vector<long long> &dist) {
    dist.assign(n + 1, LLONG_MAX);
    dist[start] = 0;
    for (int i = 0; i < n - 1; i++) {
        for (auto &[u, v, w] : edges) {
            if (dist[u] != LLONG_MAX && dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
        }
    }
    // one extra pass: if any distance can still improve, a negative
    cycle exists
    for (auto &[u, v, w] : edges) {
        if (dist[u] != LLONG_MAX && dist[u] + w < dist[v]) return false;
    }
    // negative cycle
    return true;
}
```

Why $V-1$ passes? The shortest path between any two vertices visits at most $V-1$ edges (more than that would revisit a vertex, which is never optimal without a negative cycle). Each full pass guarantees at least one more edge of the optimal path is correctly relaxed, so $V-1$ passes are enough to settle every shortest path. Complexity: $O(V \times E)$ — noticeably slower than Dijkstra, used specifically when negative weights are possible.

9.2 Floyd-Warshall

In one sentence: compute shortest paths between every pair of vertices at once, by considering each vertex k as a possible intermediate stop on the path between every pair (i, j) .

```

const long long INF = 1e18;
vector<vector<long long>> dist(n + 1, vector<long long>(n + 1,
INF));
for (int i = 1; i <= n; i++) dist[i][i] = 0;
// initialize dist[u][v] = weight for every given edge u -> v

for (int k = 1; k <= n; k++) // intermediate vertex
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (dist[i][k] < INF && dist[k][j] < INF)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);

```

Complexity: $O(V^3)$ time, $O(V^2)$ space — only practical for V roughly up to a few hundred. In exchange, it gives all-pairs shortest paths in one pass, handles negative edges (but not negative cycles, which it can also detect via a negative $\text{dist}[i][i]$ afterward), and is very short to code correctly under time pressure.

Algorithm	Handles negative edges?	Complexity	Use when
BFS	N/A (unweighted)	$O(V + E)$	All edges the same weight
0-1 BFS	No	$O(V + E)$	Edge weights are only 0 or 1
Dijkstra	No	$O((V+E) \log V)$	Non-negative weights, single source
Bellman-Ford	Yes (detects negative cycles)	$O(V \times E)$	Negative weights possible, single source
Floyd-Warshall	Yes	$O(V^3)$	All-pairs shortest paths, V is small

Chapter 10 — Minimum Spanning Tree: Kruskal and Prim

In one sentence: a Minimum Spanning Tree (MST) is the subset of edges that connects every vertex using the least total edge weight,

with no cycles — exactly $n-1$ edges for n vertices.

10.1 Kruskal's algorithm

Sort all edges by weight ascending; add each edge if its two endpoints are not already connected (checked and merged with DSU, Chapter 11). This is a direct greedy: always take the cheapest edge that doesn't create a cycle.

```
// requires a DSU (Disjoint Set Union) implementation – see
// Chapter 11
long long kruskalMST(int n, vector<array<int,3>> &edges) { //
{weight, u, v}
    sort(edges.begin(), edges.end()); // ascending by weight (first
field)
    DSU dsu(n);
    long long totalWeight = 0;
    int edgesUsed = 0;
    for (auto &[w, u, v] : edges) {
        if (dsu.find(u) != dsu.find(v)) { // adding this edge doesn't
form a cycle
            dsu.unite(u, v);
            totalWeight += w;
            edgesUsed++;
        }
    }
    return (edgesUsed == n - 1) ? totalWeight : -1; // -1 if graph
is disconnected
}
```

Complexity: $O(E \log E)$ for the sort, plus $O(E \alpha(V))$ for the DSU operations (α is the inverse Ackermann function — effectively constant). Overall $O(E \log E)$.

10.2 Prim's algorithm

Grow the MST one vertex at a time: start from any vertex, and repeatedly add the cheapest edge that connects a vertex already in the tree to one that isn't — the same min-heap “best candidate” pattern as Dijkstra, but tracking edge weight to reach a vertex rather than distance from the source.

```

long long primMST(int start, int n, vector<vector<pair<int,int>>>
&adj) {
    vector<bool> inMST(n + 1, false);
    priority_queue<pair<int,int>, vector<pair<int,int>>,
greater<pair<int,int>>> pq;
    pq.push({0, start}); // {edge weight to reach this vertex,
vertex}
    long long totalWeight = 0;
    int verticesUsed = 0;
    while (!pq.empty() && verticesUsed < n) {
        auto [w, u] = pq.top(); pq.pop();
        if (inMST[u]) continue;
        inMST[u] = true;
        totalWeight += w;
        verticesUsed++;
        for (auto [v, weight] : adj[u]) {
            if (!inMST[v]) pq.push({weight, v});
        }
    }
    return (verticesUsed == n) ? totalWeight : -1;
}

```

Complexity: $O(E \log V)$ with a binary heap. Kruskal tends to be simpler to reach for when the edge list is already the natural representation (e.g. edges given directly in the input); Prim tends to be simpler when starting from an adjacency list on a dense graph.

✦ Why greedy works here

Kruskal and Prim rely on the cut property: for any partition of the vertices into two non-empty sets, the cheapest edge crossing that partition must belong to some MST. Both algorithms repeatedly apply this fact — Kruskal implicitly (by always taking the globally cheapest safe edge), Prim explicitly (by always taking the cheapest edge crossing the current tree-vs-rest partition).

Chapter 11 — Disjoint Set Union (DSU)

In one sentence: DSU (also called Union-Find) maintains a collection of disjoint sets and answers “are u and v in the same set?” and “merge the sets containing u and v ”, both in almost $O(1)$ time.

```
struct DSU {
    vector<int> parent, rank_;
    DSU(int n) : parent(n + 1), rank_(n + 1, 0) {
        iota(parent.begin(), parent.end(), 0); // parent[i] = i initially
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]); // path
        compression
        return parent[x];
    }
    void unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return;
        if (rank_[a] < rank_[b]) swap(a, b); // union by rank
        parent[b] = a;
        if (rank_[a] == rank_[b]) rank_[a]++;
    }
};
```

11.1 Why path compression and union by rank matter

Without either optimization, repeated unite calls can build a long chain, making `find()` $O(n)$ in the worst case. Path compression flattens the chain every time `find()` is called, so future lookups are near-instant. Union by rank (or by size) always attaches the shorter tree under the taller one, preventing chains from forming in the first place. Together, both optimizations bring the amortized cost of each operation down to $O(\alpha(n))$ — the inverse Ackermann function, which is at most 4 or 5 for any n that fits in memory, so it's treated as effectively $O(1)$ in complexity analysis.

⚠ Common mistake

Implementing `unite()` without union by rank/size still passes small test cases (since `find()` with path compression alone is often fast enough in practice), but can degrade badly on adversarial inputs. Always include both optimizations — they cost almost no extra code.

11.2 What DSU recognizes

- Counting connected components: start with n components, decrement by 1 on every successful `unite()` (one that actually merged two different sets).
- Kruskal's MST (Chapter 10): checking “would this edge create a cycle?” is exactly “are these two endpoints already in the same DSU set?”
- “Are these two elements connected, given a stream of union operations?” — the direct clue phrase for DSU.
- Offline connectivity queries where unions only ever get added, never removed.

Practice — Chapter 11

Problem 13.5 — Number of Provinces [Level 2]

Hint 1: A “province” is exactly a connected component.

Approach: Union every pair of directly-connected cities with DSU; count the number of distinct roots at the end (or track a live counter, decremented per successful `unite`).

Complexity: $O(n^2 \alpha(n))$ for an $n \times n$ adjacency matrix input.

Chapter 12 — Advanced Graph Topics

12.1 Strongly Connected Components (SCC) — Kosaraju's algorithm

In one sentence: a Strongly Connected Component is a maximal group of vertices in a directed graph where every vertex can reach every other vertex in the group.

Kosaraju's algorithm finds all SCCs in three steps: (1) run DFS on the original graph, recording each vertex's finish time (exactly like DFS-based topological sort); (2) reverse every edge in the graph; (3) run DFS again on the reversed graph, processing vertices in decreasing finish-time order — each DFS tree produced in this second pass is exactly one SCC.

```
vector<vector<int>> adj, radj; // radj = graph with every edge
reversed
vector<bool> vis;
stack<int> finishOrder;

void dfs1(int u) {
    vis[u] = true;
    for (int v : adj[u]) if (!vis[v]) dfs1(v);
    finishOrder.push(u);
}

void dfs2(int u, vector<int> &component) {
    vis[u] = true;
    component.push_back(u);
    for (int v : radj[u]) if (!vis[v]) dfs2(v, component);
}

// main: run dfs1 from every unvisited vertex, reset vis, then
pop finishOrder
// and run dfs2 from every still-unvisited vertex to collect each
SCC.
```

Complexity: $O(V + E)$ — two DFS passes over the graph and its reverse, each linear. Kosaraju's is preferred in this book for being conceptually simple to reconstruct from scratch in a contest; Tarjan's algorithm finds the same SCCs in a single DFS pass using discovery times and a low-link value, and is worth learning as a faster-to-code alternative once Kosaraju's feels solid.

12.2 Bridges and articulation points

A bridge is an edge whose removal disconnects the graph (increases the number of connected components). An articulation point (cut vertex) is a vertex whose removal does the same. Both are found with a single DFS that tracks two values per vertex: its discovery time ($disc[u]$) and its low-link value ($low[u]$, the earliest discovery time reachable from u 's subtree using at most one back edge).

```

vector<int> disc, low;
vector<bool> vis;
int timer = 0;

void dfsBridges(int u, int parent, vector<vector<int>> &adj,
vector<pair<int,int>> &bridges) {
    vis[u] = true;
    disc[u] = low[u] = timer++;
    for (int v : adj[u]) {
        if (v == parent) continue; // skip the edge back to the
immediate parent
        if (vis[v]) {
            low[u] = min(low[u], disc[v]); // back edge
        } else {
            dfsBridges(v, u, adj, bridges);
            low[u] = min(low[u], low[v]);
            if (low[v] > disc[u]) {
                bridges.push_back({u, v}); // v's subtree can't reach above u
            }
        }
    }
}

```

The core idea: if the best (“lowest”) vertex reachable from v 's subtree using one back edge is still no earlier than u itself ($\text{low}[v] > \text{disc}[u]$), then removing edge $u-v$ disconnects v 's subtree from the rest of the graph — that edge is a bridge. Articulation points use an almost identical DFS with a slightly different condition ($\text{low}[v] \geq \text{disc}[u]$, and special handling for the DFS root). Complexity for both: $O(V + E)$.

♦ How to recognize it

Clue phrases: “critical connections”, “which roads/cables, if removed, disconnect the network”, “single point of failure.” These map directly to bridges (edges) or articulation points (vertices).

12.3 Euler path and circuit — a preview

An Euler path visits every edge exactly once (vertices may repeat); an Euler circuit is an Euler path that returns to its starting vertex. An undirected graph has an Euler circuit if and only if every vertex has even degree and the graph is connected (ignoring isolated vertices); it

has an Euler path (but not circuit) if exactly two vertices have odd degree. This is stated here as a fact worth recognizing — full construction (Hierholzer's algorithm) is covered alongside other advanced-algorithm techniques later in this book.

Practice — Chapter 12

Problem 13.6 — Critical Connections in a Network [Level 5]

Hint 1: “Critical connection” is this book's exact bridge-finding problem, phrased as a network.

Approach: Single DFS tracking `disc[]` and `low[]`; an edge is critical exactly when `low[child] > disc[parent]`.

Complexity: $O(V + E)$.

Part XIII — Master Revision

Must Remember

- Default to an adjacency list; only use a matrix for small, dense graphs.
- BFS gives shortest paths in unweighted graphs (or 0/1-weighted graphs via 0-1 BFS); DFS is for connectivity, cycles, and ordering.
- Undirected cycle detection checks “visited neighbor $\neq$ parent”; directed cycle detection needs an onStack array — they are not interchangeable.
- A graph is bipartite iff it has no odd-length cycle; check with 2-coloring BFS/DFS.
- Topological sort only exists for a DAG; Kahn's algorithm both sorts and detects a cycle if fewer than n vertices are output.
- Dijkstra requires non-negative weights; Bellman-Ford handles negative weights and detects negative cycles; Floyd-Warshall gives all-pairs shortest paths in $O(V^3)$.
- Kruskal (sort edges + DSU) and Prim (min-heap growing one tree) both rely on the cut property to justify their greedy choice.
- DSU with path compression + union by rank/size gives near- $O(1)$ find/union — always include both optimizations.
- Bridges and articulation points come from one DFS tracking disc[] and low[]; SCCs come from two DFS passes (Kosaraju) on the graph and its reverse.

Common Mistakes Recap

Mistake	Fix
Adjacency matrix on a graph with V up to 10^5	Use an adjacency list — $O(V^2)$ memory won't fit
Recursive DFS stack overflow on a long chain graph	Use iterative DFS with an explicit stack for large V

Mistake	Fix
Parent-check cycle detection used on a directed graph	Use the onStack[] method instead for directed graphs
Running Dijkstra on a graph with negative edges	Switch to Bellman-Ford
Forgetting the stale-entry check in Dijkstra's pop loop	Add <code>if (d > dist[u]) continue;</code> right after popping
DSU without path compression or union by rank	Always implement both — minimal extra code, large worst-case payoff
Treating a disconnected graph's BFS/DFS as covering all vertices	Loop over every vertex and start a fresh traversal from each unvisited one

Complexity Table — Graphs

Algorithm / Structure	Time	Space
BFS / DFS	$O(V + E)$	$O(V + E)$
0-1 BFS	$O(V + E)$	$O(V + E)$
Dijkstra (binary heap)	$O((V + E) \log V)$	$O(V + E)$
Bellman-Ford	$O(V \times E)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$
Kruskal's MST	$O(E \log E)$	$O(V + E)$
Prim's MST (binary heap)	$O(E \log V)$	$O(V + E)$
DSU (find / unite)	$O(\alpha(V))$ amortized	$O(V)$
Kosaraju's SCC	$O(V + E)$	$O(V + E)$
Bridges / articulation points	$O(V + E)$	$O(V)$

Interview-Style Questions

- 1 Why does BFS, not DFS, find the shortest path in an unweighted graph?

- 2 Why doesn't Dijkstra's algorithm work with negative edge weights?
- 3 Why are $V-1$ relaxation passes enough for Bellman-Ford to find all shortest paths?
- 4 Why does the DSU cycle check in Kruskal's algorithm correctly prevent cycles, using only `find()` and `unite()`?
- 5 Why do path compression and union by rank together give DSU near-constant-time operations?
- 6 What is the cut property, and how do both Kruskal's and Prim's algorithms rely on it?
- 7 Why does 0-1 BFS use a deque instead of a plain queue or a priority queue?

Cheat Sheet — Part XIII

```
// Representation: adjacency list (default choice)
vector<vector<int>> adj(n + 1); // unweighted
vector<vector<pair<int,int>>> wadj(n + 1); // weighted:
{neighbor, weight}

// Which shortest-path algorithm?
// unweighted -> BFS  $O(V + E)$ 
// weights are 0 or 1 -> 0-1 BFS (deque)  $O(V + E)$ 
// non-negative weights -> Dijkstra (min-heap)  $O((V+E) \log V)$ 
// negative weights OK -> Bellman-Ford  $O(V * E)$ 
// all-pairs, small V -> Floyd-Warshall  $O(V^3)$ 

// MST: Kruskal (sort edges + DSU) or Prim (min-heap growing one
tree)
// Connectivity / "same group?" queries -> DSU with path
compression + union by rank
// Directed-cycle check / dependency order -> Kahn's algorithm
(topological sort)
// "Critical edge/vertex" -> bridges / articulation points via
disc[]/low[] DFS
// "Groups that fully reach each other" in a directed graph ->
Kosaraju's SCC
```

 Coming up in Part XIV

Part XIV — Greedy Algorithms formalizes the “local choice that turns out to be globally optimal” reasoning already used informally for Kruskal and Prim in this Part: the exchange argument, interval scheduling, fractional knapsack, and — just as importantly — concrete examples of when greedy looks tempting but actually fails.